Client-Server Architecture (4 Marks – Exam Answer)

Definition: **Client-Server Architecture** is a network architecture in which a **client** (user device or application) sends a **request** to a **server** over a network. The **server processes the request** and sends back the required **response** to the client. This model is widely used for web applications, email systems, online banking, and cloud services.

Main Two Components

1. Client

- The client is the **user's computer, mobile device, or application**.
- It sends requests to the server.
- It displays the response received from the server.
- Example: Web browser (Chrome, Firefox), Mobile App.

2. Server

- The server is a **powerful computer or software** that provides services and resources.
- It receives client requests, processes them, accesses the database if needed, and sends the response.
- Example: Web Server, Database Server.

1. Client: A device or application that sends requests to the server and displays the received results to users.
2. Server: A central system that processes client requests, manages resources, and returns appropriate responses.
3. Network: Provides the communication channel between clients and servers, enabling reliable data transfer.
4. Middleware: Acts as a bridge between components by handling services such as authentication, messaging, and logging.
5. Database: Stores, organizes, and manages application data while ensuring security and consistency.
6. User Interface (UI): The front-end component through which users interact with the application and view results.
7. Application Logic: Contains the business rules and processing logic that handle requests and coordinate data flow between components.

Hardware and Software

Hardware: **Client Hardware:** PC, Laptop, Smartphone

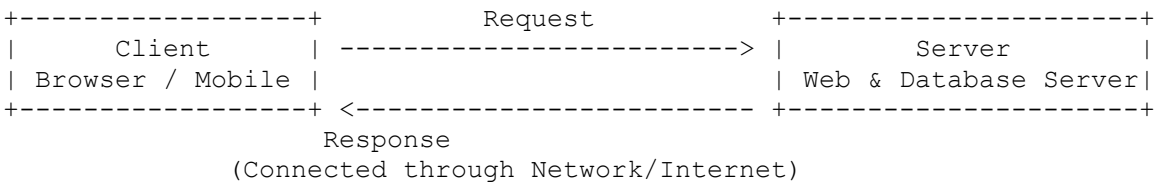
- **Server Hardware:** High-performance server computer with CPU, RAM, Storage

Software: **Client Software:** Web Browser, Mobile Application

- **Server Software:** Web Server (Apache, Nginx), Database (MySQL), Application Server

Software

Simple Diagram



Tasks in Client-Server Architecture

1. **Client sends a request** (e.g., login or open a webpage).
2. **Network** transfers the request to the server.
3. **Server processes** the request and accesses the database if required.
4. **Server sends the response** back to the client.
5. **Client displays** the received information to the user.

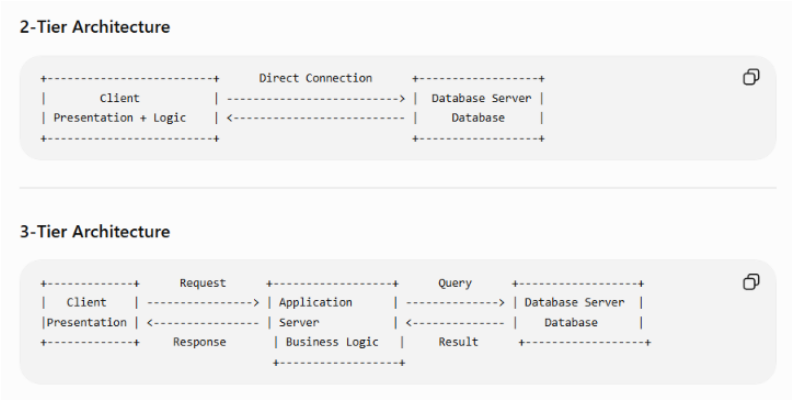
Overload Situation

- When **too many clients send requests at the same time**, the server may become **overloaded**.
- This can cause:
  - Slow response time
  - Delayed processing
  - Server crashes or temporary unavailability
- To handle overload, organizations use **load balancing**, **multiple servers**, or **cloud scaling**.

Difference Between 2-Tier and 3-Tier Architecture

2-Tier Architecture:
A client-server architecture where the **client communicates directly with the database server**. The client contains both the **presentation layer** and **business logic**.

3-Tier Architecture:
A client-server architecture where an **intermediate (application) layer** is placed between the client and the database. The **client handles only the presentation**, while the **business logic** is processed in the application server.



Feature	2-Tier Architecture	3-Tier Architecture
Layers	2 layers (Client + Database)	3 layers (Client + Application + Database)
Business Logic	Stored in the client	Stored in the application server (intermediate layer)
Client Role	Handles presentation and business logic	Handles only presentation (UI)
Database Access	Client accesses database directly	Client accesses database through application server
Scalability	Low	High
Security	Lower (direct database access)	Better (database protected by application server)
Maintainability	Harder to update	Easier to maintain and update
Performance	Good for small systems	Better for large enterprise systems

Two-Tier Database Architecture	Three-Tier Database Architecture
It is a Client-Server Architecture.	It is a Web-based application.
In two-tier, the application logic is either buried inside the user interface on the client or within the database on the server (or both).	In three-tier, the application logic or process resides in the middle-tier, it is separated from the data and the user interface.
Two-tier architecture consists of two layers : Client Tier and Database (Data Tier).	Three-tier architecture consists of three layers : Client Layer, Business Layer and Data Layer.
It is easy to build and maintain.	It is complex to build and maintain.
Two-tier architecture runs slower.	Three-tier architecture runs faster.
It is less secured as client can communicate with database directly.	It is secured as client is not allowed to communicate with database directly.

Two-Tier Database Architecture	Three-Tier Database Architecture
It results in performance loss whenever the users increase rapidly.	It results in performance loss whenever the system is run on Internet but gives more performance than two-tier architecture.
Example - Contact Management System created using MS-Access or Railway Reservation System, etc.	Example - Designing registration form which contains text box, label, button or a large website on the Internet, etc.

- **2-Tier:** Client ↔ Database, direct connection, client has presentation + business logic.
- **3-Tier:** Client → Application Server → Database.
- **Intermediate Layer = Application Server.**
- **Business Logic Layer** is in the application server.
- **Client handles only Presentation Layer.**
- **3-Tier provides better Scalability, Security, and Maintainability.**

Four types of servers in distributed systems

- **Web Servers**: Handle HTTP/HTTPS requests from clients, deliver static web pages, and route dynamic requests to backend systems.
- A**pplication Servers**: Run the core business logic, process active user data, and act as the middle layer between front-end web clients and back-end storage
- **Database Servers**: Store, retrieve, and manage structured data across distributed nodes, ensuring query performance and data synchronization.
- **Proxy Servers**: Act as intermediaries between clients and internal servers, managing traffic load, caching data, and enhancing security.

Role of Network and Data Communication in Client-Server Environment

In a **client-server environment**, the **network** is the communication medium that connects the client and server, while **data communication** is the process of sending and receiving data between them. The client sends a **request** through the network, and the server receives it, processes it, and returns a **response** back to the client.

The **network** allows devices to stay connected over LAN, WAN, or the Internet. It makes it possible for users to access shared resources such as files, web pages, databases, printers, and applications from a remote server.

Data communication ensures that information is transferred correctly, quickly, and reliably between client and server. It uses rules and protocols such as TCP/IP to control how data is sent, received, and delivered without errors.

Main roles

- Connects client and server
- Transfers requests and responses
- Supports sharing of data and resources
- Ensures reliable and secure communication

Simple flow

```
Client  --->  Network/Data Communication  --->  Server
Client  <---  Network/Data Communication  <---  Server
```

Network and data communication act as the vital physical and logical bridge in a client-server environment. They provide the transmission paths and rules that allow client devices to send requests and servers to return responses, making real-time data exchange possible

Fundamental Principles of Distributed Systems in Client-Server Architecture

- **Resource sharing**: Clients use shared resources from the server, such as files, databases, and applications.
- **Communication**: Client and server exchange data through a network using requests and responses.
- **Concurrency**: Many clients can connect to the same server at the same time.
- **Scalability**: The system can grow by adding more clients or more servers when demand increases.
- **Fault tolerance**: If one part fails, other parts may still continue working, so the system is more reliable.
- **Transparency**: Users do not need to know where the server is located; the system works like one unit.

In simple client-server form

- Client sends a **request**
- Server processes it
- Server sends back a **response**
- Network connects them

Fault Tolerance in Distributed Systems

Fault tolerance is super important in designing distributed systems because it helps keep the system running even when things go wrong, like if a computer breaks or the network has problems. Here are some main ways to handle faults in distributed systems:

- **Replication**: Making copies of data or tasks on different computers so if one fails, there's still a backup. This can be done with data, processing, or services.
- **Redundancy**: Keeping extra copies of important stuff like hardware, software, or data so if something breaks, there's a backup ready to take over. This helps avoid downtime and keeps the system running smoothly.
- **Error Detection and Recovery**: Having tools in place to spot when something goes wrong and fix it before it causes big problems. This might involve checking if everything's okay, diagnosing issues, and taking steps to get things back on track.

Q2:

4 Key Functions of the Client Operating System in a Client-Server Environment

- **User Interface (UI)**
  Provides the screen, menus, and input methods that allow the user to interact with the system easily.
- **Local Resource Management**
  Manages the client's own resources such as the **hard disk, memory, keyboard, printer, and files**.
- **Network Stack Implementation (TCP/IP)**
  Handles network communication by sending and receiving data over the network using protocols like **TCP/IP**.
- **Application and Process Management**
  Runs client applications, manages processes, and controls how programs use CPU and memory.

• **User interface** – lets the user interact with the computer through GUI/menus/windows.
• **Local resource management** – manages the client's disk, memory, input/output devices, and other local hardware. "Hard drive" is one part of this, but the broader term is better.
• **Network stack implementation (TCP/IP)** – handles network communication so the client can send requests and receive responses from the server.
• **Application/process support** – runs client applications and helps them operate while connected to the network.

Difference Between Network OS and Server OS

Feature	Network OS	Server OS
Meaning	An operating system that helps computers communicate and share resources over a network.	An operating system installed on a server to provide services to client computers.
Main purpose	Manages network connections, file sharing, printer sharing, and communication between devices.	Provides services such as web hosting, database access, authentication, and application hosting.
Users	Used in networked computers and workstations.	Used on dedicated server machines.
Role	Focuses on connecting and controlling devices in a network.	Focuses on serving many clients at the same time.
Complexity	Usually less powerful than a server OS.	More powerful and more secure because it handles heavy workloads.
Security	Basic to moderate network security.	Strong security and centralized control for users and resources.

Network OS examples:

- Novell NetWare
- Windows NT
- UNIX/Linux-based network systems

Server OS examples:

- Windows Server
- Red Hat Enterprise Linux Server
- Ubuntu Server
- Solaris

- **Network OS** = manages network communication and sharing.
- **Server OS** = provides services from a server to many clients.

Q3:

TP (Transaction Processing) Monitor

A **TP monitor** is middleware software that manages and controls **transaction processing** in a distributed system. It sits between the **client** and the **server/database** and makes sure transactions are completed correctly, safely, and efficiently.

Role of TP Monitor in Client-Server / Distributed Systems

- **Routes client requests** to the correct service or server.
- **Coordinates distributed transactions** across multiple systems.
- **Manages transaction execution** so all parts of the transaction work together.
- **Ensures ACID properties**:
  - **Atomicity** – all parts succeed or all fail
  - **Consistency** – data remains correct

    - **Isolation** – transactions do not interfere
    - **Durability** – committed changes are saved
- **Handles commit and rollback** if a failure happens.
- **Improves reliability and performance** in distributed transaction processing.

Simple diagram

```
Client  --->  TP Monitor  --->  Server / Database
   <---       response     <---
```

The **TP monitor** acts as a **controller and router** for client requests in a distributed transaction system. It helps in **routing requests, managing transactions, and ensuring reliable processing**.

differentiate between client side and server-side programming, provide one example for each

The primary difference between client-side and server-side programming is **where the code is executed**: client-side programming runs directly on the **user's web browser**, while server-side programming runs on a **remote web server**

Key Differences At A Glance

Feature	Client-Side Programming	Server-Side Programming
Execution Location	User's web browser (e.g., Chrome, Safari).	Remote backend application server.
Primary Focus	User interface (UI) and interactivity.	Data processing and business logic.
Database Access	Cannot directly connect to database servers.	Directly communicates with databases.
Primary Languages	JavaScript, HTML, CSS.	Python, PHP, Java, Ruby, C#.
Source Code Visibility	Visible to the user via browser tools.	Hidden and secure on the server.

Examples

1. Client-Side Example: Form Validation

- **Scenario**: A user fills out a registration form but forgets to include the "@" symbol in their email address.

- **How it works**: A client-side script written in JavaScript instantly checks the format as soon as the user clicks "Submit". The browser blocks the submission and displays a red error message without reloading or sending data across the internet.

2. Server-Side Example: User Authentication

- **Scenario**: A user enters their username and password on a login page and clicks "Sign In".
- **How it works**: The browser sends these credentials over the internet to the backend web server. A server-side language like Python or PHP securely hashes the password, queries the central database to check if the credentials match, and safely logs the user into their account.

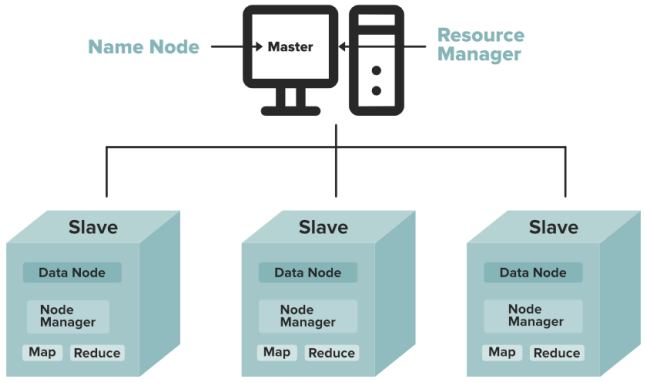
Describe the architecture of hdfs(hard distributed file system) and name it is two component(name node,

HDFS Architecture

HDFS (Hadoop Distributed File System) is a distributed storage system used to store very large files across multiple machines. It follows a **master-slave architecture**. In HDFS, files are split into large blocks and stored on different machines to provide **fault tolerance** and **high availability**.

Two Main Components of HDFS

1. **NameNode**
   - The master node
   - Stores metadata such as file names, directory structure, permissions, and block locations
   - Does not store the actual file data
2. **DataNode**
   - The worker node
   - Stores the actual data blocks
   - Sends block reports and heartbeats to the NameNode



Explain two advantages of distributed system over traditional local file system

1. High Availability
A distributed system stores data or services on more than one machine. So, if one computer is not working, other machines can still provide the data or service. This means users can access the system with less interruption than in a local file system, where everything depends on one computer.

2. Fault Tolerance
Distributed systems can continue working even when one part fails. Data is often replicated across multiple nodes, so if one node crashes, another copy is still available. In a traditional local file system, failure of the local machine can cause data loss or system downtime.

3. Scalability

A distributed system can easily be expanded by adding more computers (nodes) when the workload or amount of data increases. This improves performance and supports more users. In contrast, a traditional local file system is limited by the resources of a single computer, making it difficult and expensive to scale.

3.5 what is the purpose of file replication (give 4 with brief explain)

File replication is the process of creating and storing **multiple copies of the same file** on different servers or storage nodes in a distributed system.

1. High Availability
Multiple copies of a file ensure that users can still access the file even if one server or node becomes unavailable.

2. Fault Tolerance
If one server or disk fails, another copy of the file is available, preventing data loss and keeping the system operational.

3. Improved Performance
Users can access the nearest or least busy copy of the file, reducing access time and improving read speed.

4. Data Backup and Reliability
Having multiple copies protects data against hardware failures, corruption, or accidental deletion, making the system more reliable.

Q 4:

What is Phased Migration?

Phased migration is a migration strategy where a system is **moved from the old system to the new system gradually in multiple stages**, instead of replacing everything at once. During this period, both the old and new systems may run simultaneously, reducing risk and minimizing downtime.

Techniques Needed During Phased Migration

1. Bidirectional Synchronization

- Data is synchronized between the **old system** and the **new system**.
- Any changes made in one system are automatically updated in the other.
- This keeps both systems consistent during migration.

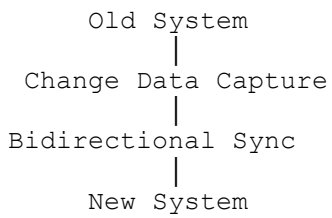
2. Change Data Capture (CDC)

- Detects and captures changes (Insert, Update, Delete) made in the source database.
- Sends those changes to the new system in real time or near real time.
- Ensures the new database stays up to date without copying all data repeatedly.

Advantages of Phased Migration

- Reduces migration risk.
- Minimizes system downtime.
- Allows gradual testing before full migration.
- Easier rollback if problems occur.

Simple Diagram



Describe Two Common Security Threats in a Distributed Client-Server System and How They Can Be Mitigated (4 Marks)

1. Denial of Service (DoS) Attack

Threat:
A **Denial of Service (DoS)** attack floods the server with a large number of **fake requests**, overloading the server. As a result, legitimate users cannot access the service.

Mitigation:

- Use **firewalls** to filter malicious traffic.
- Implement **rate limiting** to limit the number of requests from a client.
- Use **load balancing** to distribute traffic across multiple servers.
- Deploy **Intrusion Detection/Prevention Systems (IDS/IPS)** to detect and block attacks.

2. Data Interception (Eavesdropping / Man-in-the-Middle Attack)

Threat:
Attackers intercept data transmitted between the client and server, allowing them to steal sensitive information such as usernames, passwords, or financial data.

Mitigation:

- Use **SSL/TLS (Secure Sockets Layer / Transport Layer Security)** to encrypt data during transmission.
- Use **HTTPS** instead of HTTP.
- Use **digital certificates** to verify the identity of the server.

Security Threat	Mitigation
Denial of Service (DoS)	Firewalls, Rate Limiting, Load Balancing, IDS/IPS
Data Interception (MITM/Eavesdropping)	SSL/TLS encryption, HTTPS, Digital Certificates

Explain two client development issues using from building the distributed system

1. Network Latency and Unreliability

Issue:
Communication between the client and server depends on the network. Slow networks, high latency, or connection failures can delay requests or interrupt communication.

How the client should handle it:

- Implement **timeouts** and **retry mechanisms**.
- Display **loading indicators** and meaningful error messages.
- Cache data locally when appropriate to reduce repeated requests.

2. Version Compatibility

Issue:
Different clients may use **different versions** of the application while connecting to different server versions. This can cause errors if features or APIs are not compatible.

How the client should handle it:

- Maintain **backward compatibility** in APIs.

- Use **API versioning** (e.g., `/api/v1`, `/api/v2`).
- Encourage users to update to the latest client version when needed.

Client Development Issue	How to Handle It
Network latency and unreliability	Use timeouts, retries, caching, and error handling.
Version compatibility	Use API versioning and backward compatibility.

4.4 what is the purpose of DMZ is a network security and where is place?

What is a DMZ?

A **DMZ (Demilitarized Zone)** is a **physically or logically separated subnetwork** that contains **public-facing services**, such as web servers, mail servers, and DNS servers. It acts as a **buffer zone** between the Internet and the internal trusted network (LAN).

Purpose of a DMZ

1. Protects the Internal Network

The DMZ prevents direct access from the Internet to the internal LAN, improving security.

2. Hosts Public Services

It stores **external-facing services** such as:

- Web Server
- Mail Server
- DNS Server
  These services can be accessed by Internet users without exposing the internal network.

3. Reduces Security Risks

If a server in the DMZ is compromised, attackers are isolated from the internal network.

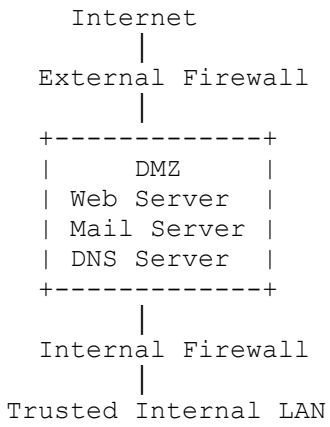
4. Adds an Extra Layer of Security

The DMZ works with firewalls to filter and monitor incoming and outgoing traffic.

Where is a DMZ Placed?

The **DMZ is placed between the Internet and the trusted internal LAN**, and it is usually **isolated by two firewalls** (or one firewall with three network interfaces).

Simple Diagram



Why is Password-Based Authentication Problematic in a Distributed Environment? (4 Marks)

Password-based authentication can create security risks in distributed systems because users access multiple services over a network.

1. Password Reuse Across Systems

Problem:
Many users reuse the **same password** for multiple systems or services. If one service is hacked and the password is stolen, attackers can use the same password to access other accounts.

Result:
A single security breach can compromise multiple systems and user accounts.

2. Network Transmission Vulnerability

Problem:
Passwords are sent over the network during login. If the connection is not secure, attackers can intercept passwords through **packet sniffing**, **man-in-the-middle (MITM)** attacks, or **brute-force attacks**.

Result:
Attackers can steal passwords and gain unauthorized access to the system.

Mitigation: Use **SSL/TLS (HTTPS)** to encrypt password transmission and use **multi-factor authentication (MFA)**.

Q: 5

Explain Two Key Challenges in Managing Distributed Systems (4 Marks)

Managing a distributed system is challenging because it consists of multiple computers (nodes) working together over a network.

1. Monitoring and Observability

Challenge:
A distributed system runs on many servers, making it difficult to monitor the health, performance, and status of all components.

Explanation:
Administrators need monitoring tools to track system performance, detect errors, collect logs, and identify the source of problems quickly.

2. Fault Tolerance and Recovery

Challenge:
Servers, networks, or storage devices may fail unexpectedly in a distributed system.

Explanation:
The system must detect failures, recover automatically, and continue providing services with minimal downtime using techniques such as redundancy, replication, and failover.

What are the Potential Impacts of Future Trends in Distributed Systems? (4 Marks)

1. Edge Computing

Impact:
Edge computing moves data processing from **centralized cloud servers** to the **edge of the network** (closer to users and IoT devices).

Benefits:

- Reduces **network latency** (faster response time).
- Reduces **bandwidth usage** because less data is sent to the cloud.
- Improves performance for **IoT devices**, autonomous vehicles, and real-time applications.

2. Quantum Computing

Impact:
Quantum computers have the potential to **break current cryptographic algorithms** such as **RSA** and **ECC**, which are widely used to secure distributed systems.

Benefits/Challenges:

- Existing security protocols may become vulnerable.
- Organizations will need to **redesign security protocols**.
- Systems must migrate to **Post-Quantum Cryptography (PQC)** to protect future communications.

what is the load balancing in distributed system and why it is important?

What is Load Balancing?

Load balancing is the process of **distributing incoming client requests across multiple servers or resources** instead of sending all requests to a single server.

This ensures that no single server becomes overloaded and that the workload is shared evenly.

Why is Load Balancing Important?

1. Improves Performance

By distributing requests among multiple servers, load balancing prevents one server from becoming overloaded and **reduces response time**.

2. Ensures High Availability

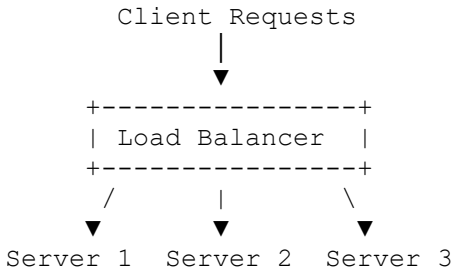
If one server fails, the load balancer automatically **redirects client requests to other available servers**, keeping the service running.

3. Supports Scalability

New servers can be **added easily** to the resource pool as demand increases, allowing the system to handle more users without major changes.

4. Increases Reliability

Sharing the workload across multiple servers reduces the risk of system failure and provides more reliable service.



Describe two or 4 type of failures that can occur in distributed computing environment

A distributed computing environment consists of multiple computers (nodes) connected through a network. Different types of failures can occur during operation.

1. Network Failure

Description:
Occurs when communication between clients and servers or between nodes is interrupted due to network problems, causing delays or loss of data.

2. Node Crash Failure

Description:
Occurs when a server or node suddenly stops working because of a system crash or power failure. The node cannot process requests until it is restarted.

3. Hardware Failure

Description:
Occurs when physical components such as the **CPU, hard disk, memory (RAM), or network card** fail, causing the affected node to stop functioning properly.

4. Software Failure

Description:
Occurs due to software bugs, application crashes, operating system errors, or incorrect configurations, which may cause incorrect results or service interruption.

Failure Type	Description
Network Failure	Communication between nodes is interrupted.
Node Crash Failure	A server or node unexpectedly stops working.
Hardware Failure	Failure of physical components such as CPU, RAM, or disk.
Software Failure	Failure caused by bugs, crashes, or configuration errors.

why is scalability considered a critical factor in the design of distributed client server system

Scalability is the ability of a distributed client-server system to **handle increasing workloads or users by adding more resources** without significantly affecting performance.

1. Handles Growing Workloads

As the number of users and requests increases, scalability allows the system to **add more servers or resources** to maintain good performance and fast response times.

2. Supports Horizontal Scaling

Distributed systems can use **horizontal scaling**, where **additional commodity servers (nodes)** are added instead of upgrading one powerful server. This is **more cost-effective** and provides better flexibility than **vertical scaling**.

3. Improves Performance

By distributing the workload across multiple servers, the system avoids bottlenecks and reduces response time.

4. Ensures High Availability

Scalable systems can continue to provide services even during high traffic by sharing the workload among multiple servers.